



RTL Design (Lab Report – Week 1 & 2)

Submitted by:

Mavia Ahmed

Submitted to:

Sir Khallil Rehman

Table of Contents

(RTL Design Lab)

Week#	Date	Lab Tasks
01	15-June-2026 17-June-2026 20-June-2026	1. Implement AND gate using Behavioral, Data Flow and Structural Models 2. Implement OR gate using Behavioral, Data Flow and Structural Models 3. Implement XOR gate using Behavioral, Data Flow and Structural Models 4. Design and run the code for the given schematic on software 5. Implement Half Adder 6. Implement Half Subtractor 7. Implement Full Adder 8. Implement Full Subtractor 9. Design and run the code for the given schematic on software 10. Implement a 4x1 MUX 11. Implement a BCD to 7-Segment Display 12. Implement a 4 input Priority Encoder 13. Implement a 3-to-8 Decoder 14. Implement a 4-to-2 Encoder 15. Implement an 8-bit binary Multiplier
02	22-June-2026 24-June-2026 27-June-2026	1. Implement T-Flipflop in System Verilog. 2. Implement D-Flipflop in System Verilog. 3. Implement SR-Flipflop in System Verilog. 4. Implement JK-Flipflop in System Verilog. 5. Convert a D-Flipflop into a T-Flipflop (Make a T-Flipflop using a D-Flipflop) 6. Convert a T-Flipflop into a D-Flipflop (Make a D-Flipflop using a T-Flipflop) 7. Implement a 4-bit Up/Down Counter (Synchronous) in System Verilog. 8. Implement a 4-bit Up-Counter (Ripple/Asynchronous) in System Verilog. 9. Implement a BCD Counter in System Verilog. 10. Implement a Mod N Counter in System Verilog. 11. Implement a Clock Frequency Divider (100Mhz to 25Mhz) in System Verilog. 12. Implement a Johnson Counter in System Verilog. 13. Implement a Ring Counter in System Verilog.

Table of Contents

(RTL Design Lab)

		14. Implement a Universal Shift Register (SISO, SIPO, PISO, PIPO) in System Verilog. 15. Implement a Bi-Directional Shift Register (SISIO, SIPO) in System Verilog. 16. Implement an Edge Detector in System Verilog. 17. Implement a 32-bit Multiplier (Pipelined) in System Verilog. 18. Implement a 32-bit ALU in System Verilog.
03	14-July-2026 15-July-2026 18-July-2026	1. Implement a 1011 Sequence Detector using FSM in SystemVerilog. 2. Implement Traffic Lights using FSM in SystemVerilog. 3. Implement a Vending Machine using FSM in SystemVerilog. 4. Implement an 8-bit FIFO Buffer in SystemVerilog.
04		

GitHub Repository Link:

<https://github.com/mavia369/USTP-RTL-Design-Labs.git>

Week No. 1

Task 1:

Objective:

- Implement **AND** gate using **Behavioral**, **Data Flow** and **Structural** Models.

Design Code:

AND Behavioral

```

1 //Behavioral Model
2
3 module and_beh(
4     input logic a, b,
5     output logic y
6 );
7
8 always @(*) begin
9     y = a & b;
10 end
11
12 endmodule

```

AND Data Flow

```

1 //Data Flow Model
2
3 module and_data(
4     input logic a,
5     input logic b,
6     output logic y
7 );
8
9 assign y = a & b;
10
11 endmodule

```

AND Structural

```

1 // //Structural Model
2
3 module and_struct(
4     input logic a,b,
5     output logic y
6 );
7
8     and G1(y, a, b);
9
10 endmodule

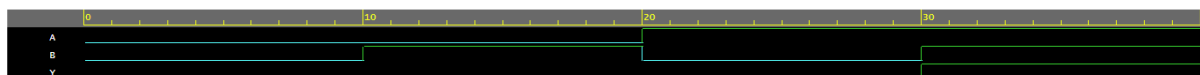
```

Test Bench Code:

```

1 //Test Bench
2 module and_tb;
3
4 logic A, B;
5 logic Y;
6
7 and_beh dut(
8     .a(A),
9     .b(B),
10    .y(Y)
11 );
12
13 initial begin
14     A=0; B=0; #10;
15     A=0; B=1; #10;
16     A=1; B=0; #10;
17     A=1; B=1; #10;
18
19     $finish;
20 end
21
22 initial begin
23     $dumpfile("tb_and.vcd");
24     $dumpvars(0, and_tb);
25 end
26
27 endmodule

```

Output:**AND Behavioral****AND Data Flow****AND Structural**

Task 2:

Objective:

- Implement **OR** gate using **Behavioral**, **Data Flow** and **Structural** Models.

Design Code:

OR Behavioral

```
1 //Behavioral Model
2 module or_beh(
3     input logic a,b,
4     output logic y
5 );
6
7 always @(*) begin
8     y = a | b;
9 end
10
11 endmodule
```

OR Data Flow

```
1 //Data Flow Model
2
3 module or_data(
4     input logic a,b,
5     output logic y
6 );
7
8 assign y = a | b;
9
10 endmodule
```

OR Structural

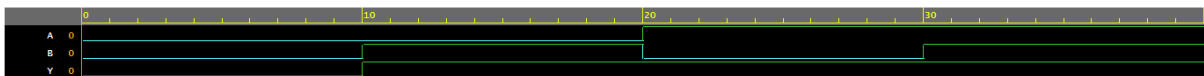
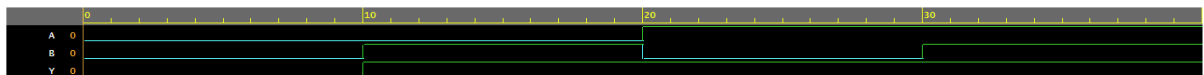
```
1 //Structural Model
2
3 module or_struct(
4     input logic a,b,
5     output logic y
6 );
7
8 or G1(y, a, b);
9
10 endmodule
```

Test Bench Code:

```

1 //Test Bench
2 module or_tb;
3
4 logic A, B;
5 logic Y;
6
7 or_beh dut(
8     .a(A),
9     .b(B),
10    .y(Y)
11 );
12
13 initial begin
14     A=0; B=0; #10;
15     A=0; B=1; #10;
16     A=1; B=0; #10;
17     A=1; B=1; #10;
18
19     $finish;
20 end
21
22 initial begin
23     $dumpfile("tb_or.vcd");
24     $dumpvars(0, or_tb);
25 end
26
27 endmodule

```

Output:**OR Behavioral****OR Data Flow****OR Structural**

Task 3:

Objective:

- Implement **XOR** gate using **Behavioral**, **Data Flow** and **Structural** Models.

Design Code:

XOR Behavioral

```
1 //Behavioral Model
2
3 module xor_beh(
4     input logic a,b,
5     output logic y
6 );
7
8 always @(*) begin
9     y = a ^ b;
10 end
11
12 endmodule
```

XOR Data Flow

```
1 //Data Flow Model
2
3 module xor_data(
4     input logic a,b,
5     output logic y
6 );
7
8 assign y = a ^ b;
9
10 endmodule
```

XOR Structural

```
1 //Structural Model
2
3 module xor_struct(
4     input logic a,b,
5     output logic y
6 );
7
8 xor G1(y, a, b);
9
10 endmodule
```


Test Bench Code:

```

1 //Test Bench
2 module xor_tb;
3
4 logic A, B;
5 logic Y;
6
7 xor_beh dut(
8     .a(A),
9     .b(B),
10    .y(Y)
11 );
12
13 initial begin
14     A=0; B=0; #10;
15     A=0; B=1; #10;
16     A=1; B=0; #10;
17     A=1; B=1; #10;
18
19     $finish;
20 end
21
22 initial begin
23     $dumpfile("tb_xor.vcd");
24     $dumpvars(0, xor_tb);
25 end
26
27 endmodule

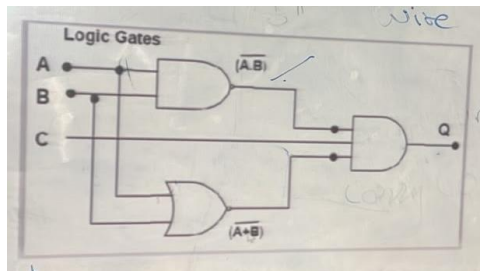
```

Output:**XOR Behavioral****XOR Data Flow****XOR Structural**

Task 4:

Objective:

- Design and run the code for the below schematic on software.



Design Code:

```

1 module given_schematic(
2     input logic a,b,c,
3     output logic q
4 );
5
6 always_comb begin
7     q = ~(a&b)&(c)&~(a|b);
8 end
9
10 endmodule

```

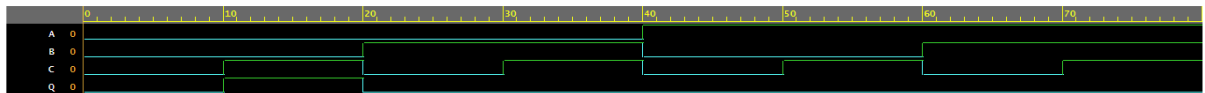
Test Bench Code:

```

1 module given_schematic_tb;
2
3 logic A, B, C;
4 logic Q;
5
6 given_schematic dut (
7     .a(A),
8     .b(B),
9     .c(C),
10    .q(Q)
11 );
12
13 initial begin
14     $dumpfile("given_schematic.vcd");
15     $dumpvars(0, given_schematic_tb);
16
17     A=0; B=0; C=0; #10;
18     A=0; B=0; C=1; #10;
19     A=0; B=1; C=0; #10;
20     A=0; B=1; C=1; #10;
21     A=1; B=0; C=0; #10;
22     A=1; B=0; C=1; #10;
23     A=1; B=1; C=0; #10;
24     A=1; B=1; C=1; #10;
25
26     $finish;
27 end
28
29 endmodule

```

Output:



Task 5:

Objective:

Implement **Half Adder**.

Design Code:

```

1 module half_adder(
2     input logic a, b,
3     output logic sum, carry
4 );
5
6 always_comb begin
7     sum = a ^ b;
8     carry = a & b;
9 end
10
11 endmodule

```

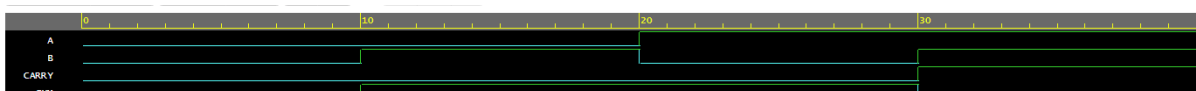
Test Bench Code:

```

1 module half_adder_tb;
2
3 logic A, B;
4 logic SUM, CARRY;
5
6 half_adder dut(
7     .a(A),
8     .b(B),
9     .sum(SUM),
10    .carry(CARRY)
11 );
12
13 initial begin
14     $dumpfile("half_adder.vcd");
15     $dumpvars(0, half_adder_tb);
16
17     A=0; B=0; #10;
18     A=0; B=1; #10;
19     A=1; B=0; #10;
20     A=1; B=1; #10;
21
22     $finish;
23 end
24
25 endmodule

```

Output:



Task 6:

Objective:

Implement **Half Subtractor**.

Design Code:

```

1 module half_subtractor(
2     input logic a, b,
3     output logic diff, borrow
4 );
5
6 always_comb begin
7     diff  = a ^ b;
8     borrow = (~a) & b;
9 end
10
11 endmodule

```

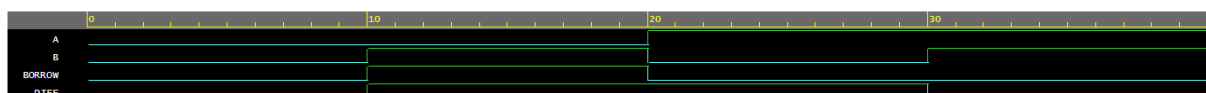
Test Bench Code:

```

1 module half_subtractor_tb;
2
3 logic A, B;
4 logic DIFF, BORROW;
5
6 half_subtractor dut(
7     .a(A),
8     .b(B),
9     .diff(DIFF),
10    .borrow(BORROW)
11 );
12
13 initial begin
14     $dumpfile("half_subtractor.vcd");
15     $dumpvars(0, half_subtractor_tb);
16
17     A=0; B=0; #10;
18     A=0; B=1; #10;
19     A=1; B=0; #10;
20     A=1; B=1; #10;
21
22     $finish;
23 end
24
25 endmodule

```

Output:



Task 7:

Objective:

Implement **Full Adder**.

Design Code:

```

1 module full_adder(
2     input logic a, b, cin,
3     output logic sum, cout
4 );
5
6 always_comb begin
7     sum = a ^ b ^ cin;
8     cout = (a & b) | (b & cin) | (a & cin);
9 end
10
11 endmodule

```

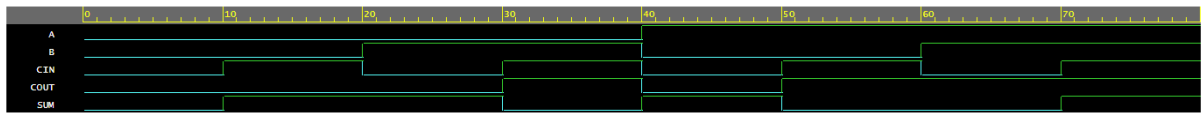
Test Bench Code:

```

1 module full_adder_tb;
2
3 logic A, B, CIN;
4 logic SUM, COUT;
5
6 full_adder dut(
7     .a(A),
8     .b(B),
9     .cin(CIN),
10    .sum(SUM),
11    .cout(COUT)
12 );
13
14 initial begin
15     $dumpfile("full_adder.vcd");
16     $dumpvars(0, full_adder_tb);
17
18     A=0; B=0; CIN=0; #10;
19     A=0; B=0; CIN=1; #10;
20     A=0; B=1; CIN=0; #10;
21     A=0; B=1; CIN=1; #10;
22     A=1; B=0; CIN=0; #10;
23     A=1; B=0; CIN=1; #10;
24     A=1; B=1; CIN=0; #10;
25     A=1; B=1; CIN=1; #10;
26
27     $finish;
28 end
29
30 endmodule

```

Output:



Task 8:

Objective:

Implement **Full Subtractor**.

Design Code:

```

1 module full_subtractor(
2     input logic a, b, bin,
3     output logic diff, bout
4 );
5
6 always_comb begin
7     diff = a ^ b ^ bin;
8     bout = (~a & b) | (~a & bin) | (b & bin);
9 end
10
11 endmodule

```

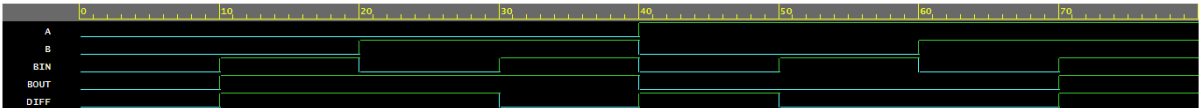
Test Bench Code:

```

1 module full_subtractor_tb;
2
3 logic A, B, BIN;
4 logic DIFF, BOUT;
5
6 full_subtractor dut(
7     .a(A),
8     .b(B),
9     .bin(BIN),
10    .diff(DIFF),
11    .bout(BOUT)
12 );
13
14 initial begin
15     $dumpfile("full_subtractor.vcd");
16     $dumpvars(0, full_subtractor_tb);
17
18     A=0; B=0; BIN=0; #10;
19     A=0; B=0; BIN=1; #10;
20     A=0; B=1; BIN=0; #10;
21     A=0; B=1; BIN=1; #10;
22     A=1; B=0; BIN=0; #10;
23     A=1; B=0; BIN=1; #10;
24     A=1; B=1; BIN=0; #10;
25     A=1; B=1; BIN=1; #10;
26
27     $finish;
28 end
29
30 endmodule

```

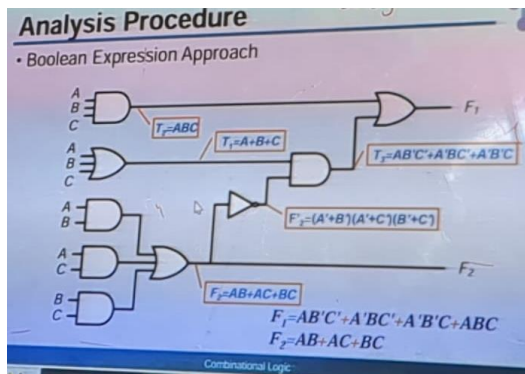

Output:



Task 9:

Objective:

- Design and run the code for the below schematic on software.



Design Code:

```

1 module circuit(
2     input logic A, B, C,
3     output logic F1, F2
4 );
5
6 logic T1, T2, F2_bar, T3;
7
8 always_comb begin
9     T1    = A & B & C;
10    T2    = A | B | C;
11
12    F2     = (A & B) | (A & C) | (B & C);
13
14    F2_bar = (~A | ~B) &
15             (~A | ~C) &
16             (~B | ~C);
17
18    T3     = (A & ~B & ~C) |
19             (~A & B & ~C) |
20             (~A & ~B & C);
21
22    F1     = (A & ~B & ~C) |
23             (~A & B & ~C) |
24             (~A & ~B & C) |
25             (A & B & C);
26 end
27
28 endmodule

```

Test Bench Code:

```

1 module circuit_tb;
2
3 logic A, B, C;
4 logic F1, F2;
5
6 circuit dut(
7     .A(A),
8     .B(B),
9     .C(C),
10    .F1(F1),
11    .F2(F2)
12 );
13
14 initial begin
15     $dumpfile("circuit.vcd");
16     $dumpvars(0, circuit_tb);
17
18     A=0; B=0; C=0; #10;
19     A=0; B=0; C=1; #10;
20     A=0; B=1; C=0; #10;
21     A=0; B=1; C=1; #10;
22     A=1; B=0; C=0; #10;
23     A=1; B=0; C=1; #10;
24     A=1; B=1; C=0; #10;
25     A=1; B=1; C=1; #10;
26
27     $finish;
28 end
29
30 endmodule

```

Output:

Task 10:

Objective:

Implement a 4x1 MUX.

Design Code:

```

1 module mux4to1(
2     input logic [3:0] i,
3     input logic [1:0] sel,
4     output logic out
5 );
6
7     always_comb begin
8         case(sel)
9             2'b00: out = i[0];
10            2'b01: out = i[1];
11            2'b10: out = i[2];
12            2'b11: out = i[3];
13            default: out = 0;
14        endcase
15    end
16
17 endmodule

```

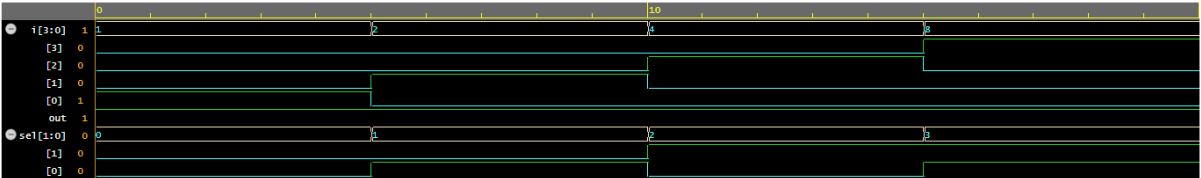
Test Bench Code:

```

1 module tb_mux4to1;
2
3     logic [3:0] i;
4     logic [1:0] sel;
5     logic out;
6
7     mux4to1 dut (
8         .i(i),
9         .sel(sel),
10        .out(out)
11    );
12
13    initial begin
14        $dumpfile("mux4to1_tb.vcd");
15        $dumpvars(0, tb_mux4to1);
16
17        i = 4'b0001; sel = 2'b00; #5;
18        i = 4'b0010; sel = 2'b01; #5;
19        i = 4'b0100; sel = 2'b10; #5;
20        i = 4'b1000; sel = 2'b11; #5;
21        $finish;
22    end
23
24 endmodule

```

Output:



Task 11:

Objective:

Implement a BCD to 7-Segment Display.

Design Code:

```

1 module bcd_to_7seg(
2     input  logic [3:0] bcd,
3     output logic [6:0] seg
4 );
5
6     always_comb begin
7         case (bcd)
8             4'b0000: seg = 7'b1111110;
9             4'b0001: seg = 7'b0110000;
10            4'b0010: seg = 7'b1101101;
11            4'b0011: seg = 7'b1111001;
12            4'b0100: seg = 7'b0110011;
13            4'b0101: seg = 7'b1011011;
14            4'b0110: seg = 7'b1011111;
15            4'b0111: seg = 7'b1110000;
16            4'b1000: seg = 7'b1111111;
17            4'b1001: seg = 7'b1111011;
18            default: seg = 7'b0000000;
19        endcase
20    end
21
22 endmodule

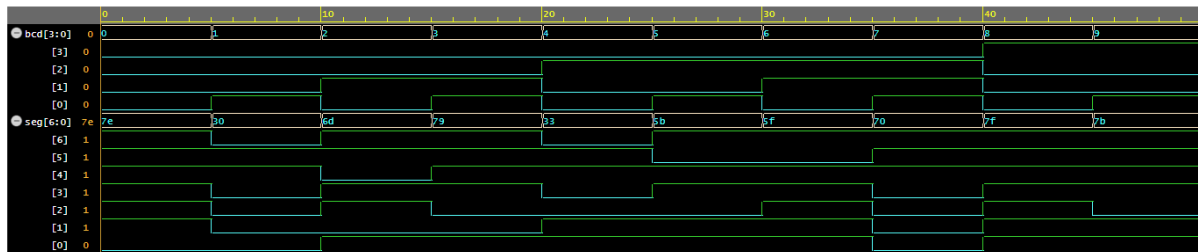
```

Test Bench Code:

```

1 module tb_bcd_to_7seg;
2
3     logic [3:0] bcd;
4     logic [6:0] seg;
5
6     bcd_to_7seg dut(
7         .bcd(bcd),
8         .seg(seg)
9     );
10
11     initial begin
12         bcd = 4'b0000; #5;
13         bcd = 4'b0001; #5;
14         bcd = 4'b0010; #5;
15         bcd = 4'b0011; #5;
16         bcd = 4'b0100; #5;
17         bcd = 4'b0101; #5;
18         bcd = 4'b0110; #5;
19         bcd = 4'b0111; #5;
20         bcd = 4'b1000; #5;
21         bcd = 4'b1001; #5;
22
23         $finish;
24     end
25
26     initial begin
27         $dumpfile("bcd_to_7seg_tb.vcd");
28         $dumpvars(0, tb_bcd_to_7seg);|
29     end
30
31 endmodule

```

Output:

Task 12:

Objective:

Implement a 4 input **Priority Encoder**.

Design Code:

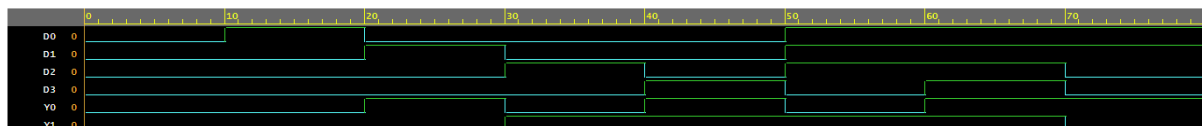
```
1 module priority_encoder_4to2(  
2     input logic D3, D2, D1, D0,  
3     output logic Y1, Y0  
4 );  
5  
6 always_comb begin  
7     if (D3) begin  
8         Y1 = 1;  
9         Y0 = 1;  
10    end  
11    else if (D2) begin  
12        Y1 = 1;  
13        Y0 = 0;  
14    end  
15    else if (D1) begin  
16        Y1 = 0;  
17        Y0 = 1;  
18    end  
19    else begin  
20        Y1 = 0;  
21        Y0 = 0;  
22    end  
23 end  
24 |  
25 endmodule
```


Test Bench Code:

```

1 module priority_encoder_4to2_tb;
2
3 logic D3, D2, D1, D0;
4 logic Y1, Y0;
5
6 priority_encoder_4to2 dut(
7     .D3(D3),
8     .D2(D2),
9     .D1(D1),
10    .D0(D0),
11    .Y1(Y1),
12    .Y0(Y0)
13 );
14
15 initial begin
16     $dumpfile("priority_encoder.vcd");
17     $dumpvars(0, priority_encoder_4to2_tb);
18
19     D3=0; D2=0; D1=0; D0=0; #10;
20     D3=0; D2=0; D1=0; D0=1; #10;
21     D3=0; D2=0; D1=1; D0=0; #10;
22     D3=0; D2=1; D1=0; D0=0; #10;
23     D3=1; D2=0; D1=0; D0=0; #10;
24
25     // Priority checks
26     D3=0; D2=1; D1=1; D0=1; #10;
27     D3=1; D2=1; D1=1; D0=1; #10;
28     D3=0; D2=0; D1=1; D0=1; #10;
29
30     $finish;
31 end
32
33 endmodule

```

Output:

Task 13:

Objective:

Implement a 3-to-8 Decoder.

Design Code:

```

1 module decoder_3to8(
2     input logic A, B, C,
3     output logic Y0, Y1, Y2, Y3,
4         Y4, Y5, Y6, Y7
5 );
6
7 always_comb begin
8     Y0=0; Y1=0; Y2=0; Y3=0;
9     Y4=0; Y5=0; Y6=0; Y7=0;
10
11     case ({A,B,C})
12         3'b000: Y0 = 1;
13         3'b001: Y1 = 1;
14         3'b010: Y2 = 1;
15         3'b011: Y3 = 1;
16         3'b100: Y4 = 1;
17         3'b101: Y5 = 1;
18         3'b110: Y6 = 1;
19         3'b111: Y7 = 1;
20     endcase
21 end
22
23 endmodule

```

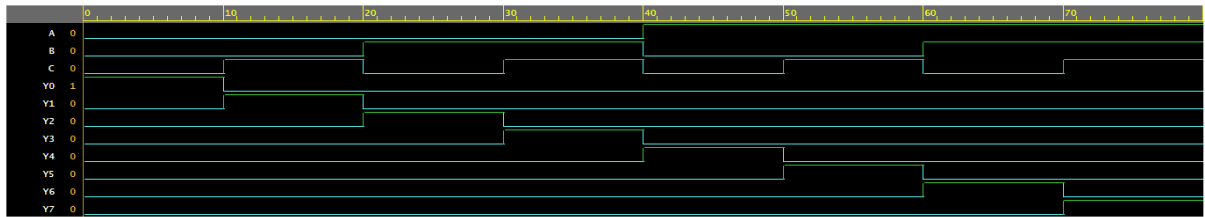
Test Bench Code:

```

1 module decoder_3to8_tb;
2
3 logic A, B, C;
4 logic Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7;
5
6 decoder_3to8 dut(
7     .A(A), .B(B), .C(C),
8     .Y0(Y0), .Y1(Y1), .Y2(Y2), .Y3(Y3),
9     .Y4(Y4), .Y5(Y5), .Y6(Y6), .Y7(Y7)
10 );
11
12 initial begin
13     $dumpfile("decoder_3to8.vcd");
14     $dumpvars(0, decoder_3to8_tb);
15
16     A=0; B=0; C=0; #10;
17     A=0; B=0; C=1; #10;
18     A=0; B=1; C=0; #10;
19     A=0; B=1; C=1; #10;
20     A=1; B=0; C=0; #10;
21     A=1; B=0; C=1; #10;
22     A=1; B=1; C=0; #10;
23     A=1; B=1; C=1; #10;
24
25     $finish;
26 end
27
28 endmodule

```

Output:



Task 14:

Objective:

Implement a 4-to-2 Encoder.

Design Code:

```

1 module encoder_4to2(
2     input logic D3, D2, D1, D0,
3     output logic Y1, Y0
4 );
5
6 always_comb begin
7     Y1 = D2 | D3;
8     Y0 = D1 | D3;
9 end
10
11 endmodule

```

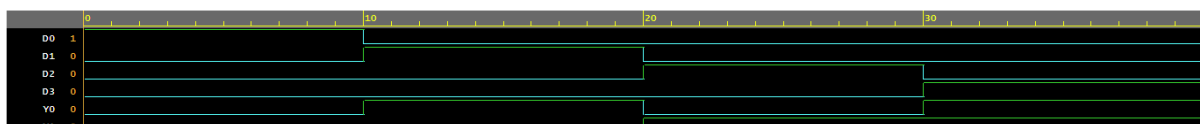
Test Bench Code:

```

1 module encoder_4to2_tb;
2
3 logic D3, D2, D1, D0;
4 logic Y1, Y0;
5
6 encoder_4to2 dut(
7     .D3(D3),
8     .D2(D2),
9     .D1(D1),
10    .D0(D0),
11    .Y1(Y1),
12    .Y0(Y0)
13 );
14
15 initial begin
16     $dumpfile("encoder_4to2.vcd");
17     $dumpvars(0, encoder_4to2_tb);
18
19     D3=0; D2=0; D1=0; D0=1; #10;
20     D3=0; D2=0; D1=1; D0=0; #10;
21     D3=0; D2=1; D1=0; D0=0; #10;
22     D3=1; D2=0; D1=0; D0=0; #10;
23
24     $finish;
25 end
26
27 endmodule

```

Output:



Task 15:

Objective:

Implement an **8-bit binary Multiplier**.

Design Code:

```

1 module multiplier_8bit(
2     input logic [7:0] A,
3     input logic [7:0] B,
4     output logic [15:0] P
5 );
6
7 integer i;
8
9 always_comb begin
10     P = 16'd0;
11
12     for (i = 0; i < 8; i = i + 1) begin
13         if (B[i])
14             P = P + (A << i);
15     end
16 end
17
18 endmodule

```

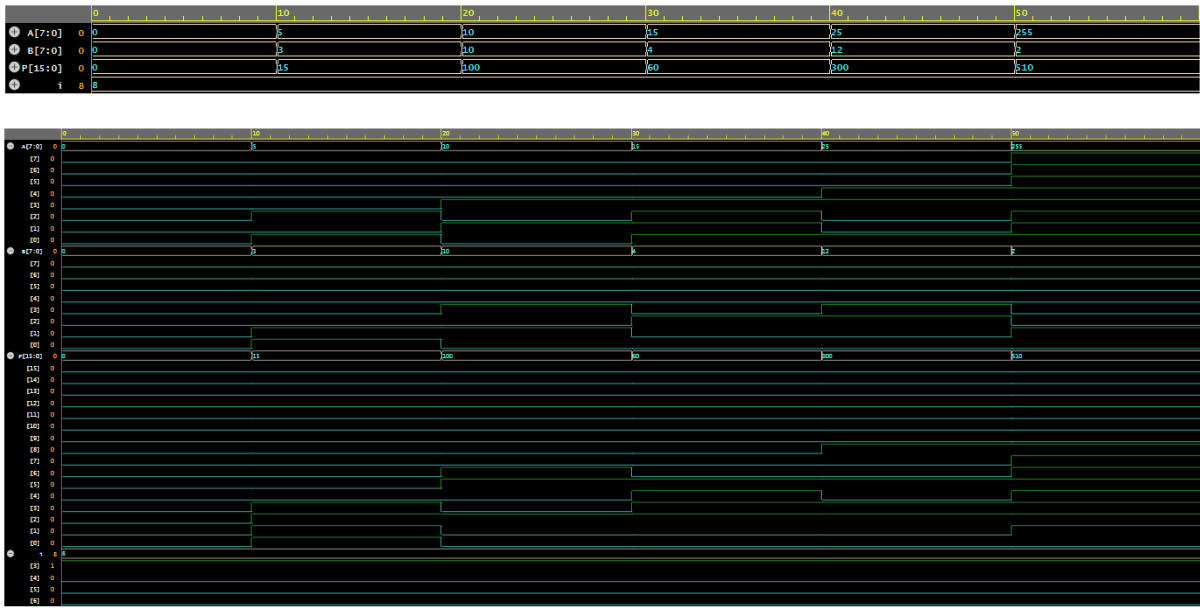
Test Bench Code:

```

1 module multiplier_8bit_tb;
2
3 logic [7:0] A, B;
4 logic [15:0] P;
5
6 multiplier_8bit dut(
7     .A(A),
8     .B(B),
9     .P(P)
10 );
11
12 initial begin
13     $dumpfile("multiplier_8bit.vcd");
14     $dumpvars(0, multiplier_8bit_tb);
15
16     A = 8'd0;    B = 8'd0;    #10;
17     A = 8'd5;    B = 8'd3;    #10;
18     A = 8'd10;   B = 8'd10;   #10;
19     A = 8'd15;   B = 8'd4;    #10;
20     A = 8'd25;   B = 8'd12;   #10;
21     A = 8'd255;  B = 8'd2;    #10;
22
23     $finish;
24 end
25
26 endmodule

```

Output:



Week No. 2

Task 1:

Objective:

Implement T-Flipflop in System Verilog.

Design Code:

```
module t_flipflop (
    input  logic clk,
    input  logic rst,
    input  logic t,
    output logic q
);

always_ff @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else if (t)
        q <= ~q;
    else
        q <= q;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module t_flipflop_tb;

    logic clk;
    logic rst;
    logic t;
    logic q;

    t_flipflop uut (
        .clk(clk),
        .rst(rst),
        .t(t),
        .q(q)
    );

endmodule
```

```

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    t   = 0;

    #10;
    rst = 0;

    #10;
    t = 0;

    #10;
    t = 1;

    #40;

    t = 0;
    #20;

    t = 1;
    #30;

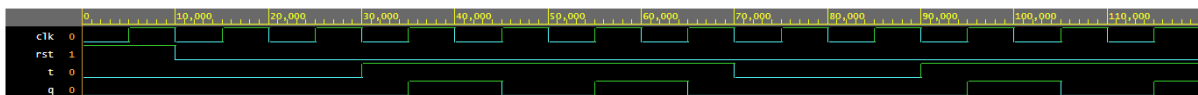
    $finish;
end

initial begin
    $dumpfile("t_flipflop.vcd");
    $dumpvars(0, t_flipflop_tb);
end

endmodule

```

Output:



Task 2:

Objective:

Implement D-Flipflop in System Verilog.

Design Code:

```
module d_flipflop(
    input logic clock, reset,
    input logic d,
    output logic q
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 0;
    else
        q <= d;
end

endmodule
```

Test Bench Code:

```
module tb_d_flipflop;

    logic clock, reset, d;
    logic q;

    d_flipflop dut(
        .clock(clock),
        .reset(reset),
        .d(d),
        .q(q)
    );

    always #5 clock = ~clock;

    initial begin
        clock = 0;
        reset = 1;
        d = 0;

        #10;
        reset = 0;

        #10 d = 1;
        #10 d = 0;
        #10 d = 1;
    end

endmodule
```

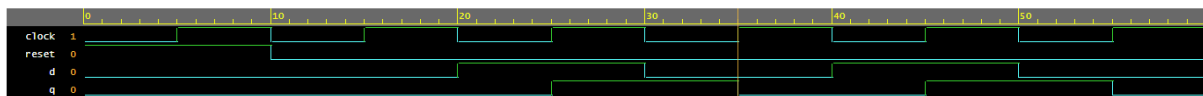
```
#10 d = 0;

#10;
$finish;
end

initial begin
    $dumpfile("d_flipflop.vcd");
    $dumpvars(0,tb_d_flipflop);
end

endmodule
```

Output:



Task 3:

Objective:

Implement SR-Flipflop in System Verilog.

Design Code:

```
module sr_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic s,
    input  logic r,
    output logic q
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else begin
        case ({s, r})
            2'b00: q <= q;
            2'b01: q <= 1'b0;
            2'b10: q <= 1'b1;
            2'b11: q <= 1'bx;
        endcase
    end
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_sr_flipflop;

    logic clock;
    logic reset;
    logic s;
    logic r;
    logic q;

    sr_flipflop dut (
        .clock(clock),
        .reset(reset),
        .s(s),
        .r(r),
        .q(q)
    );

endmodule
```

```

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    s = 0;
    r = 0;

    #10;
    reset = 0;

    #10;
    s = 0;
    r = 0;

    #10;
    s = 1;
    r = 0;

    #10;
    s = 0;
    r = 1;

    #10;
    s = 1;
    r = 1;

    #10;

    s = 0;
    r = 0;

    #20;

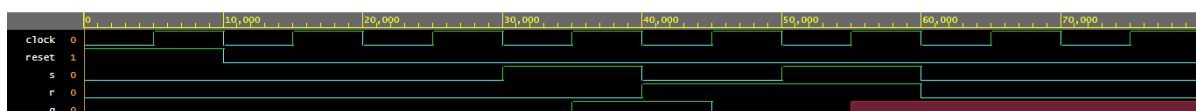
    $finish;
end

initial begin
    $dumpfile("sr_flipflop.vcd");
    $dumpvars(0, tb_sr_flipflop);
end

endmodule

```

Output:



Task 4:

Objective:

Implement JK-Flipflop in System Verilog.

Design Code:

```
module jk_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic j,
    input  logic k,
    output logic q
);

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else begin
        case ({j, k})
            2'b00: q <= q;
            2'b01: q <= 1'b0;
            2'b10: q <= 1'b1;
            2'b11: q <= ~q;
        endcase
    end
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_jk_flipflop;

    logic clock;
    logic reset;
    logic j;
    logic k;
    logic q;

    jk_flipflop dut (
        .clock(clock),
        .reset(reset),
        .j(j),
        .k(k),
        .q(q)
    );

endmodule
```

```

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    j = 0;
    k = 0;

    #10;
    reset = 0;

    #10;
    j = 0;
    k = 0;

    #10;
    j = 1;
    k = 0;

    #10;
    j = 0;
    k = 1;

    #10;
    j = 1;
    k = 1;

    #20;

    j = 0;
    k = 0;

    #20;

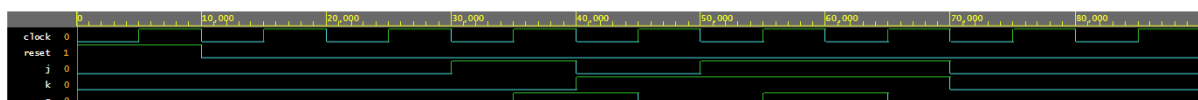
    $finish;
end

initial begin
    $dumpfile("jk_flipflop.vcd");
    $dumpvars(0, tb_jk_flipflop);
end

endmodule

```

Output:



Task 5:

Objective:

Convert a D-Flipflop into a T-Flipflop (Make a T-Flipflop using a D-Flipflop)

(Hint: Characteristic equation).

Design Code:

```
module d_to_t_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic t,
    output logic q
);

logic d;

assign d = t ^ q;

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else
        q <= d;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_d_to_t_flipflop;

logic clock;
logic reset;
logic t;
logic q;

d_to_t_flipflop dut(
    .clock(clock),
    .reset(reset),
    .t(t),
    .q(q)
);

always #5 clock = ~clock;
```

```

initial begin
    clock = 0;
    reset = 1;
    t = 0;

    #10;
    reset = 0;

    #10 t = 0;

    #10 t = 1;

    #20;

    t = 0;
    #20;

    t = 1;
    #20;

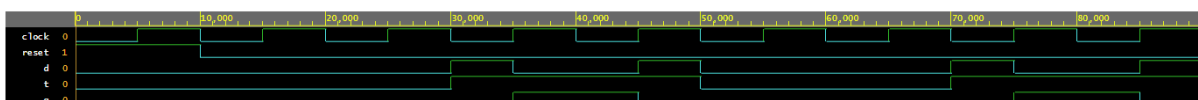
    $finish;
end

initial begin
    $dumpfile("d_to_t.vcd");
    $dumpvars(0, tb_d_to_t_flipflop);
end

endmodule

```

Output:



Task 6:

Objective:

Convert a T-Flipflop into a D-Flipflop (Make a D-Flipflop using a T-Flipflop)

(Hint: Characteristic equation).

Design Code:

```
module t_to_d_flipflop(
    input  logic clock,
    input  logic reset,
    input  logic d,
    output logic q
);

logic t;

assign t = d ^ q;

always_ff @(posedge clock or posedge reset) begin
    if (reset)
        q <= 1'b0;
    else if (t)
        q <= ~q;
    else
        q <= q;
end

endmodule
```

Test Bench Code:

```
`timescale 1ns/1ps

module tb_t_to_d_flipflop;

logic clock;
logic reset;
logic d;
logic q;

t_to_d_flipflop dut(
    .clock(clock),
    .reset(reset),
    .d(d),
    .q(q)
);

endmodule
```

```

always #5 clock = ~clock;

initial begin
    clock = 0;
    reset = 1;
    d = 0;

    #10;
    reset = 0;

    #10 d = 0;
    #10 d = 1;
    #10 d = 0;
    #10 d = 1;
    #10 d = 1;
    #10 d = 0;

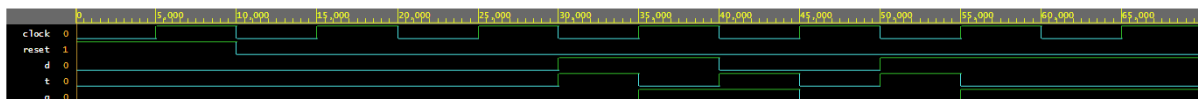
    $finish;
end

initial begin
    $dumpfile("t_to_d.vcd");
    $dumpvars(0, tb_t_to_d_flipflop);
end

endmodule

```

Output:



Task 7:

Objective:

Implement a 4-bit **Up/Down Counter (Synchronous)** in System Verilog.

Design Code:

```

1 module up_down_counter_synchronous (
2     input clock,
3     input reset,
4     input up_down,
5     output reg [3:0] count
6 );
7
8     always @(posedge clock or posedge reset) begin
9         if(reset)
10            count <= 4'b0000;
11        else if (up_down)
12            count <= count + 1;
13        else
14            count <= count -1;
15    end
16
17 endmodule

```

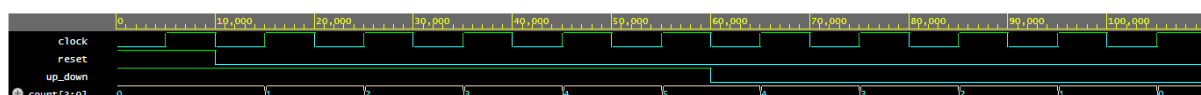
Test Bench Code:

```

1 `timescale 1ns/1ps
2
3 module tb_up_down_counter_synchronous;
4
5     reg clock;
6     reg reset;
7     reg up_down;
8     wire [3:0] count;
9
10    up_down_counter_synchronous uut (
11        .clock(clock),
12        .reset(reset),
13        .up_down(up_down),
14        .count(count)
15    );
16
17    always #5 clock = ~clock;
18
19    initial begin
20        clock = 0;
21        reset = 1;
22        up_down = 1;
23
24        #10 reset = 0;
25
26        #50;
27
28        up_down = 0;
29        #50;
30
31        $finish;
32    end
33
34    initial begin
35        $dumpfile("up_down_counter_synchronous.vcd");
36        $dumpvars(0, tb_up_down_counter_synchronous);
37    end
38
39 endmodule

```

Output:



Task 8:

Objective:

Implement a 4-bit **Up-Counter (Ripple/Asynchronous)** in System Verilog.

Design Code:

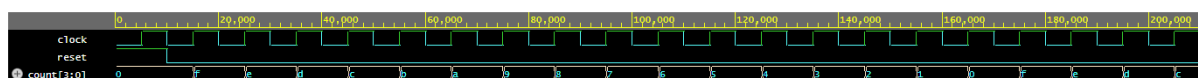
```
1 module async_up_counter (  
2     input clock,  
3     input reset,  
4     output reg [3:0] count  
5 );  
6  
7 always @(posedge clock or posedge reset) begin  
8     if (reset)  
9         count[0] <= 0;  
10    else  
11        count[0] <= ~count[0];  
12    end  
13  
14 always @(posedge count[0] or posedge reset) begin  
15     if (reset)  
16         count[1] <= 0;  
17     else  
18         count[1] <= ~count[1];  
19    end  
20  
21 always @(posedge count[1] or posedge reset) begin  
22     if (reset)  
23         count[2] <= 0;  
24     else  
25         count[2] <= ~count[2];  
26    end  
27  
28 always @(posedge count[2] or posedge reset) begin  
29     if (reset)  
30         count[3] <= 0;  
31     else  
32         count[3] <= ~count[3];  
33    end  
34  
35 endmodule
```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_async_up_counter;
4
5  reg clock;
6  reg reset;
7  wire [3:0] count;
8
9  async_up_counter uut (
10     .clock(clock),
11     .reset(reset),
12     .count(count)
13 );
14
15 always #5 clock = ~clock;
16
17 initial begin
18     clock = 0;
19     reset = 1;
20
21     #10 reset = 0;
22
23     #200;
24
25     $finish;
26 end
27
28 initial begin
29     $dumpfile("async_up_counter.vcd");
30     $dumpvars(0, tb_async_up_counter);
31 end
32
33 endmodule

```

Output:

Task 9:

Objective:

Implement a **BCD Counter** in System Verilog.

Design Code:

```

1 module bcd_counter(
2     input  logic clock,
3     input  logic reset,
4     output logic [3:0] count
5 );
6
7 always_ff @(posedge clock or posedge reset) begin
8     if (reset)
9         count <= 4'd0;
10    else if (count == 4'd9)
11        count <= 4'd0;
12    else
13        count <= count + 1'b1;
14 end
15
16 endmodule

```

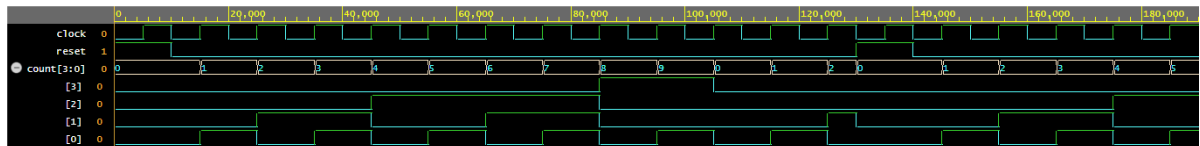
Test Bench Code:

```

1 `timescale 1ns/1ps
2
3 module tb_bcd_counter;
4
5     logic clock;
6     logic reset;
7     logic [3:0] count;
8
9     bcd_counter dut (
10         .clock(clock),
11         .reset(reset),
12         .count(count)
13     );
14
15     always #5 clock = ~clock;
16
17     initial begin
18         clock = 0;
19         reset = 1;
20
21         #10;
22         reset = 0;
23
24         #120;
25
26         reset = 1;
27         #10;
28         reset = 0;
29
30         #50;
31
32         $finish;
33     end
34
35     initial begin
36         $dumpfile("bcd_counter.vcd");
37         $dumpvars(0, tb_bcd_counter);
38     end
39
40 endmodule

```

Output:



Task 10:

Objective:

Implement a **Mod N Counter** in System Verilog.

Design Code:

```

1 module mod_n_counter #(
2     parameter N = 10
3 )
4     input  logic clock,
5     input  logic reset,
6     output logic [$clog2(N)-1:0] count
7 );
8
9 always_ff @(posedge clock or posedge reset) begin
10     if (reset)
11         count <= 0;
12     else if (count == N-1)
13         count <= 0;
14     else
15         count <= count + 1'b1;
16 end
17
18 endmodule

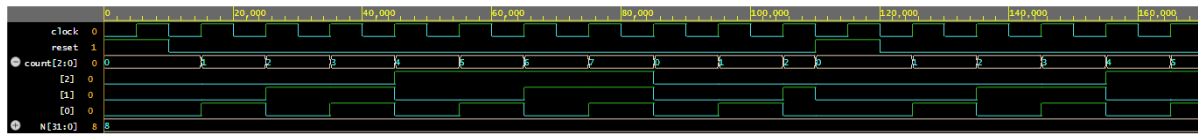
```

Test Bench Code:

```

1 `timescale 1ns/1ps
2
3 module tb_mod_n_counter;
4
5     parameter N = 8;
6
7     logic clock;
8     logic reset;
9     logic [$clog2(N)-1:0] count;
10
11     mod_n_counter #(.N(N)) dut (
12         .clock(clock),
13         .reset(reset),
14         .count(count)
15     );
16
17     always #5 clock = ~clock;
18
19     initial begin
20         clock = 0;
21         reset = 1;
22
23         #10;
24         reset = 0;
25
26         #100;
27
28         reset = 1;
29         #10;
30         reset = 0;
31
32         #50;
33
34         $finish;
35     end
36
37     initial begin
38         $dumpfile("mod_n_counter.vcd");
39         $dumpvars(0, tb_mod_n_counter);
40     end
41
42 endmodule

```


Output:

Task 11:

Objective:

Implement a **Clock Frequency Divider** (100Mhz to 25Mhz) in System Verilog.

Design Code:

```

1 module clock_divider(
2     input logic clock,
3     input logic reset,
4     output logic clk_out
5 );
6
7 logic [1:0] count;
8
9 always_ff @(posedge clock or posedge reset) begin
10     if (reset)
11         count <= 2'b00;
12     else
13         count <= count + 1'b1;
14 end
15
16 assign clk_out = count[1];
17
18 endmodule

```

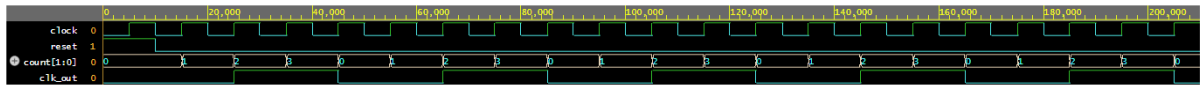
Test Bench Code:

```

1 `timescale 1ns/1ps
2
3 module tb_clock_divider;
4
5 logic clock;
6 logic reset;
7 logic clk_out;
8
9 clock_divider dut(
10     .clock(clock),
11     .reset(reset),
12     .clk_out(clk_out)
13 );
14
15 always #5 clock = ~clock;
16
17 initial begin
18     clock = 0;
19     reset = 1;
20
21     #10;
22     reset = 0;
23
24     #200;
25
26     $finish;
27 end
28
29 initial begin
30     $dumpfile("clock_divider.vcd");
31     $dumpvars(0, tb_clock_divider);
32 end
33
34 endmodule

```

Output:



Task 12:

Objective:

Implement a **Johnson Counter** in System Verilog.

Design Code:

```

1 module johnson_counter (
2     input logic clock,
3     input logic reset,
4     output logic [3:0] q
5 );
6
7     always_ff @(posedge clock or posedge reset) begin
8         if (reset)
9             q <= 4'b0000;
10        else
11            q <= {q[2:0], ~q[3]};
12    end
13
14 endmodule

```

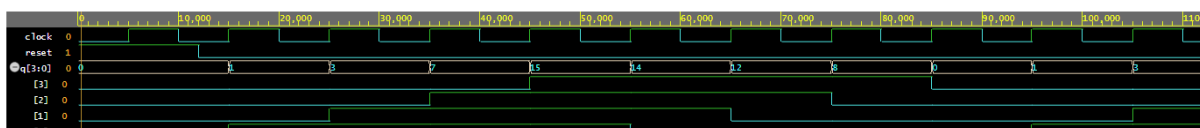
Test Bench Code:

```

1 `timescale 1ns/1ps
2
3 module tb_johnson_counter;
4
5     logic clock;
6     logic reset;
7     logic [3:0] q;
8
9     johnson_counter dut (
10         .clock(clock),
11         .reset(reset),
12         .q(q)
13     );
14
15     initial begin
16         clock = 0;
17         forever #5 clock = ~clock;
18     end
19
20     initial begin
21         reset = 1;
22
23         #12;
24         reset = 0;
25
26         #100;
27
28         $finish;
29     end
30
31     initial begin
32         $dumpfile("johnson_counter.vcd");
33         $dumpvars(0, tb_johnson_counter);
34     end
35
36 endmodule

```

Output:



Task 13:

Objective:

Implement a **Ring Counter** in System Verilog.

Design Code:

```

1 module ring_counter (
2     input  logic clock,
3     input  logic reset,
4     output logic [3:0] q
5 );
6
7     always_ff @(posedge clock or posedge reset) begin
8         if (reset)
9             q <= 4'b0001;
10        else
11            q <= {q[2:0], q[3]};
12    end
13
14 endmodule

```

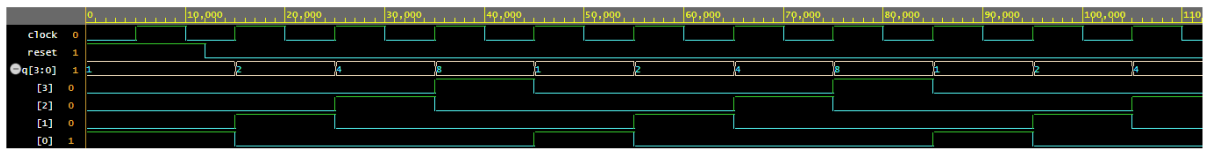
Test Bench Code:

```

1 `timescale 1ns/1ps
2
3 module tb_ring_counter;
4
5     logic clock;
6     logic reset;
7     logic [3:0] q;
8
9     ring_counter dut (
10        .clock(clock),
11        .reset(reset),
12        .q(q)
13    );
14
15    initial begin
16        clock = 0;
17        forever #5 clock = ~clock;
18    end
19
20    initial begin
21        reset = 1;
22
23        #12;
24        reset = 0;
25
26        #100;
27
28        $finish;
29    end
30
31    initial begin
32        $dumpfile("ring_counter.vcd");
33        $dumpvars(0, tb_ring_counter);
34    end
35
36 endmodule

```

Output:



Task 14:

Objective:

Implement a **Universal Shift Register (SISO, SIPO, PISO, PIPO)** in System Verilog.

Design Code:

```

1  module universal_shift_register(
2      input  logic clock,
3      input  logic reset,
4
5      input  logic [1:0] mode,
6
7      input  logic serial_left_in,
8      input  logic serial_right_in,
9
10     input  logic [3:0] parallel_in,
11
12     output logic [3:0] q,
13
14     output logic serial_left_out,
15     output logic serial_right_out
16 );
17
18 always_ff @(posedge clock or posedge reset) begin
19
20     if (reset)
21         q <= 4'b0000;
22
23     else begin
24         case (mode)
25
26             2'b00:
27                 q <= q;
28
29             2'b01:
30                 q <= {serial_left_in, q[3:1]};
31
32             2'b10:
33                 q <= {q[2:0], serial_right_in};
34
35             2'b11:
36                 q <= parallel_in;
37
38             default:
39                 q <= q;
40
41         endcase
42     end
43
44 end
45
46 assign serial_left_out  = q[3];
47 assign serial_right_out = q[0];
48
49 endmodule

```

Test Bench Code:

```
1  `timescale 1ns/1ps
2
3  module tb_universal_shift_register;
4
5  logic clock;
6  logic reset;
7
8  logic [1:0] mode;
9
10 logic serial_left_in;
11 logic serial_right_in;
12
13 logic [3:0] parallel_in;
14
15 logic [3:0] q;
16
17 logic serial_left_out;
18 logic serial_right_out;
19
20 universal_shift_register dut(
21
22     .clock(clock),
23     .reset(reset),
24
25     .mode(mode),
26
27     .serial_left_in(serial_left_in),
28     .serial_right_in(serial_right_in),
29
30     .parallel_in(parallel_in),
31
32     .q(q),
33 |
34     .serial_left_out(serial_left_out),
35     .serial_right_out(serial_right_out)
36
37 );
38
39 always #5 clock = ~clock;
40
41 initial begin
42
43     clock = 0;
```

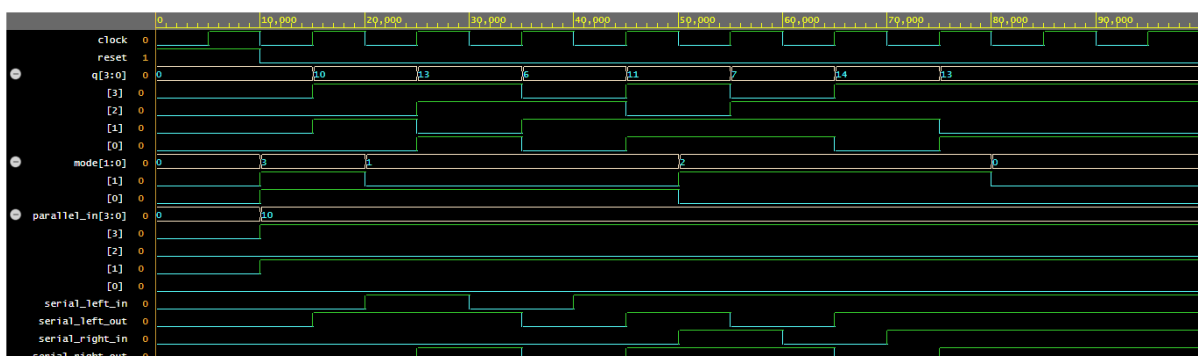


```

44     reset = 1;
45     mode = 2'b00;
46
47     serial_left_in = 0;
48     serial_right_in = 0;
49
50     parallel_in = 4'b0000;
51
52     #10;
53     reset = 0;
54
55     mode = 2'b11;
56     parallel_in = 4'b1010;
57     #10;
58
59     mode = 2'b01;
60
61     serial_left_in = 1; #10;
62     serial_left_in = 0; #10;
63     serial_left_in = 1; #10;
64
65     mode = 2'b10;
66
67     serial_right_in = 1; #10;
68     serial_right_in = 0; #10;
69     serial_right_in = 1; #10;
70
71     mode = 2'b00;
72     #20;
73
74     $finish;
75
76
77 end
78
79 initial begin
80     $dumpfile("universal_shift_register.vcd");
81     $dumpvars(0, tb_universal_shift_register);
82 end
83
84 endmodule

```

Output:



Task 15:

Objective:

Implement a **Bi-Directional Shift Register (SISIO, SIPO)** in System Verilog.

Design Code:

```
1 module bidirectional_shift_register(  
2     input  logic clock,  
3     input  logic reset,  
4     input  logic direction,  
5     input  logic serial_left_in,  
6     input  logic serial_right_in,  
7     output logic [3:0] q,  
8     output logic serial_left_out,  
9     output logic serial_right_out  
10 );  
11  
12 always_ff @(posedge clock or posedge reset) begin  
13     if (reset)  
14         q <= 4'b0000;  
15     else begin  
16         if (direction == 1'b0)  
17             q <= {serial_left_in, q[3:1]};  
18         else  
19             q <= {q[2:0], serial_right_in};  
20     end  
21 end  
22  
23 assign serial_left_out  = q[3];  
24 assign serial_right_out = q[0];  
25  
26 endmodule
```

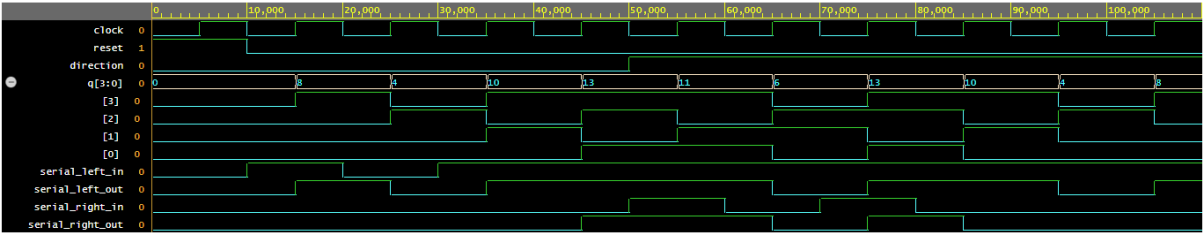
Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_bidirectional_shift_register;
4
5  logic clock;
6  logic reset;
7  logic direction;
8  logic serial_left_in;
9  logic serial_right_in;
10
11 logic [3:0] q;
12 logic serial_left_out;
13 logic serial_right_out;
14
15 bidirectional_shift_register dut(
16     .clock(clock),
17     .reset(reset),
18     .direction(direction),
19     .serial_left_in(serial_left_in),
20     .serial_right_in(serial_right_in),
21     .q(q),
22     .serial_left_out(serial_left_out),
23     .serial_right_out(serial_right_out)
24 );
25
26 always #5 clock = ~clock;
27
28 initial begin
29     clock = 0;
30     reset = 1;
31     direction = 0;
32     serial_left_in = 0;
33     serial_right_in = 0;
34
35     #10;
36     reset = 0;
37
38     direction = 0;
39     serial_left_in = 1; #10;
40     serial_left_in = 0; #10;
41     serial_left_in = 1; #10;
42     serial_left_in = 1; #10;
43
44     direction = 1;
45     serial_right_in = 1; #10;
46     serial_right_in = 0; #10;
47     serial_right_in = 1; #10;
48     serial_right_in = 0; #10;
49
50     #20;
51
52     $finish;
53 end
54
55 initial begin
56     $dumpfile("bidirectional_shift_register.vcd");
57     $dumpvars(0, tb_bidirectional_shift_register);
58 end
59
60 endmodule

```

Output:



Task 16:

Objective:

Implement an **Edge Detector** in System Verilog.

Design Code:

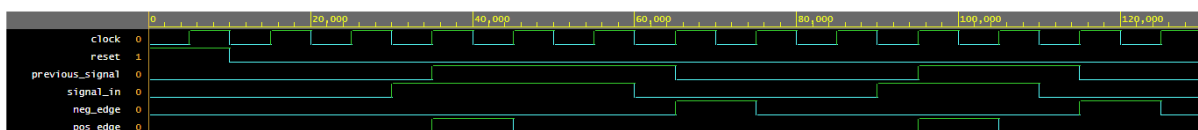
```
1 module edge_detector(  
2     input  logic clock,  
3     input  logic reset,  
4     input  logic signal_in,  
5     output logic pos_edge,  
6     output logic neg_edge  
7 );  
8  
9 logic previous_signal;  
10  
11 always_ff @(posedge clock or posedge reset) begin  
12     if (reset) begin  
13         previous_signal <= 1'b0;  
14         pos_edge       <= 1'b0;  
15         neg_edge       <= 1'b0;  
16     end  
17     else begin  
18         pos_edge <= signal_in & ~previous_signal;  
19         neg_edge <= ~signal_in & previous_signal;  
20         previous_signal <= signal_in;  
21     end  
22 end  
23  
24 endmodule
```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_edge_detector;
4
5  logic clock;
6  logic reset;
7  logic signal_in;
8  logic pos_edge;
9  logic neg_edge;
10
11  edge_detector dut(
12      .clock(clock),
13      .reset(reset),
14      .signal_in(signal_in),
15      .pos_edge(pos_edge),
16      .neg_edge(neg_edge)
17  );
18
19  always #5 clock = ~clock;
20
21  initial begin
22      clock = 0;
23      reset = 1;
24      signal_in = 0;
25      |
26      #10;
27      reset = 0;
28
29      #10 signal_in = 0;
30      #10 signal_in = 1;
31      #20 signal_in = 1;
32      #10 signal_in = 0;
33      #20 signal_in = 0;
34      #10 signal_in = 1;
35      #20 signal_in = 0;
36
37      #20;
38
39      $finish;
40  end
41
42  initial begin
43      $dumpfile("edge_detector.vcd");
44      $dumpvars(0, tb_edge_detector);
45  end
46
47  endmodule

```

Output:

Task 17:

Objective:

Implement a **32-bit Multiplier (Pipelined)** in System Verilog.

Design Code:

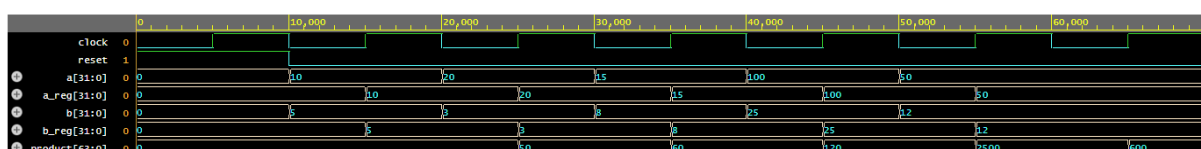
```
1 module pipelined_multiplier(  
2     input logic clock,  
3     input logic reset,  
4     input logic [31:0] a,  
5     input logic [31:0] b,  
6     output logic [63:0] product  
7 );  
8  
9 logic [31:0] a_reg;  
10 logic [31:0] b_reg;  
11  
12 always_ff @(posedge clock or posedge reset) begin  
13     if (reset) begin  
14         a_reg <= 32'd0;  
15         b_reg <= 32'd0;  
16     end  
17     else begin  
18         a_reg <= a;  
19         b_reg <= b;  
20     end  
21 end  
22  
23 always_ff @(posedge clock or posedge reset) begin  
24     if (reset)  
25         product <= 64'd0;  
26     else  
27         product <= a_reg * b_reg;  
28 end  
29  
30 endmodule
```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_pipelined_multiplier;
4
5  logic clock;
6  logic reset;
7  logic [31:0] a;
8  logic [31:0] b;
9  logic [63:0] product;
10
11  pipelined_multiplier dut(
12      .clock(clock),
13      .reset(reset),
14      .a(a),
15      .b(b),
16      .product(product)
17  );
18
19  always #5 clock = ~clock;
20
21  initial begin
22      clock = 0;
23      reset = 1;
24      a = 0;
25      b = 0;
26
27      #10;
28      reset = 0;
29
30      a = 10;    b = 5;
31      #10;
32
33      a = 20;    b = 3;
34      #10;
35
36      a = 15;    b = 8;
37      #10;
38
39      a = 100;   b = 25;
40      #10;
41
42      a = 50;    b = 12;
43      #20;
44
45      $finish;
46  end
47
48  initial begin
49      $dumpfile("pipelined_multiplier.vcd");
50      $dumpvars(0, tb_pipelined_multiplier);
51  end
52
53  endmodule

```

Output:

Task 18:

Objective:

Implement a **32-bit ALU** in System Verilog.

Design Code:

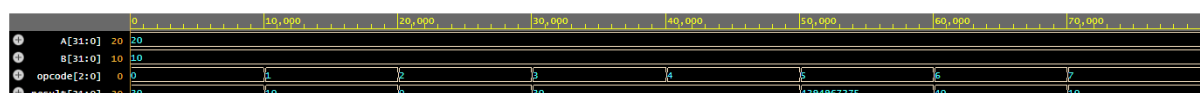
```
1 module alu_32bit(  
2     input  logic [31:0] A,  
3     input  logic [31:0] B,  
4     input  logic [2:0] opcode,  
5     output logic [31:0] result  
6 );  
7  
8 always_comb begin  
9     case (opcode)  
10        3'b000: result = A + B;  
11        3'b001: result = A - B;  
12        3'b010: result = A & B;  
13        3'b011: result = A | B;  
14        3'b100: result = A ^ B;  
15        3'b101: result = ~A;  
16        3'b110: result = A << 1;  
17        3'b111: result = A >> 1;  
18        default: result = 32'd0;  
19    endcase  
20 end  
21  
22 endmodule
```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_alu_32bit;
4
5  logic [31:0] A;
6  logic [31:0] B;
7  logic [2:0] opcode;
8  logic [31:0] result;
9
10 alu_32bit dut(
11     .A(A),
12     .B(B),
13     .opcode(opcode),
14     .result(result)
15 );
16
17 initial begin
18
19     A = 32'd20;
20     B = 32'd10;
21
22     opcode = 3'b000;
23     #10;
24
25     opcode = 3'b001;
26     #10;
27
28     opcode = 3'b010;
29     #10;
30
31     opcode = 3'b011;
32     #10;|
33
34     opcode = 3'b100;
35     #10;
36
37     opcode = 3'b101;
38     #10;
39
40     opcode = 3'b110;
41     #10;
42
43     opcode = 3'b111;
44
45     #10;
46     $finish;
47 end
48
49 initial begin
50     $dumpfile("alu_32bit.vcd");
51     $dumpvars(0, tb_alu_32bit);
52 end
53
54 endmodule

```

Output:

Week No. 3

Task 1:

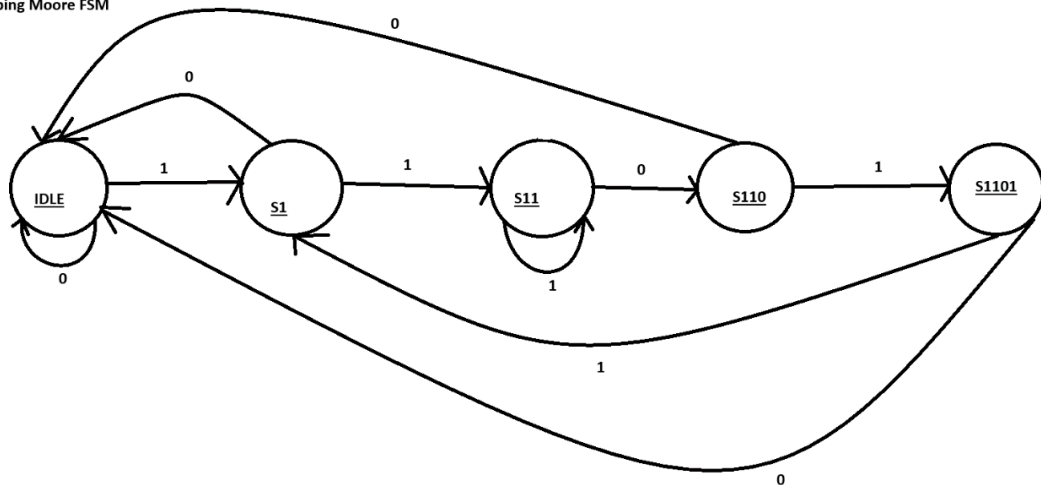
Objective:

Implement a 1011 **Sequence Detector** using FSM in SystemVerilog.

State Diagram:

Seq 1101

Non-Overlapping Moore FSM



Design Code:

```

1 module seq_detector_1101_moore (
2     input logic clk,
3     input logic rst_n,
4     input logic din,
5     output logic dout
6 );
7
8     typedef enum logic [2:0] {
9         IDLE = 3'b000,
10        S1    = 3'b001,
11        S11   = 3'b010,
12        S110  = 3'b011,
13        S1101 = 3'b100
14    } state_t;
15
16    state_t current_state, next_state;
17
18    always_ff @(posedge clk or negedge rst_n) begin
19        if (!rst_n) begin
20            current_state <= IDLE;
21        end else begin
22            current_state <= next_state;
23        end
24    end
25
26    always_comb begin
27        next_state = current_state;
28
29        case (current_state)
30            IDLE: begin
31                if (din) next_state = S1;
32                else    next_state = IDLE;
33            end
34
35            S1: begin
36                if (din) next_state = S11;
37                else    next_state = IDLE;
38            end
39
40            S11: begin
41                if (din) next_state = S11;
42                else    next_state = S110;
43            end
44
45            S110: begin
46                if (din) next_state = S1101;
47                else    next_state = IDLE;
48            end
49
50            S1101: begin
51                if (din) next_state = S1;
52                else    next_state = IDLE;
53            end
54            default: next_state = IDLE;
55        endcase
56    end
57
58    assign dout = (current_state == S1101);
59 endmodule

```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_seq_detector_1101_moore;
4
5      logic clk;
6      logic rst_n;
7      logic din;
8      logic dout;
9
10     seq_detector_1101_moore dut (
11         .clk      (clk),
12         .rst_n    (rst_n),
13         .din      (din),
14         .dout     (dout)
15     );
16
17     always #10 clk = ~clk;
18
19     task send_bit(input logic bit_in);
20     begin
21         din = bit_in;
22         @(posedge clk);
23         #1;
24     end
25 endtask
26
27 initial begin
28     clk = 0;
29     rst_n = 0;
30     din = 0;
31
32     #40;
33     rst_n = 1;
34     @(posedge clk);
35
36
37     send_bit(1);
38     send_bit(1);
39     send_bit(0);
40     send_bit(1);
41
42     send_bit(0);
43
44     send_bit(1);
45     send_bit(1);
46     send_bit(0);
47     send_bit(1);
48     send_bit(1);
49     send_bit(0);
50     send_bit(1);
51
52     send_bit(1);
53     send_bit(1);
54     send_bit(0);
55     send_bit(1);
56     send_bit(1);
57     send_bit(1);
58     send_bit(0);
59     send_bit(1);

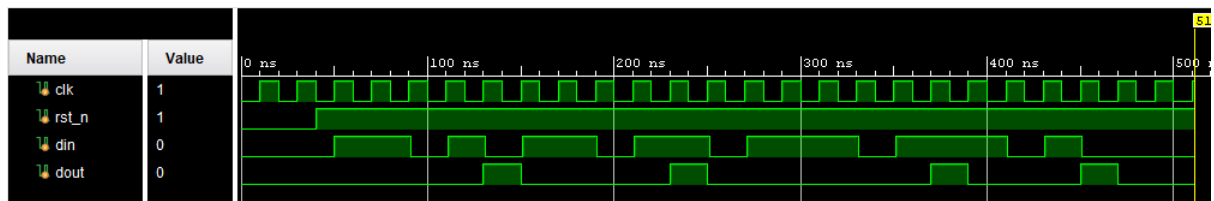
```

```

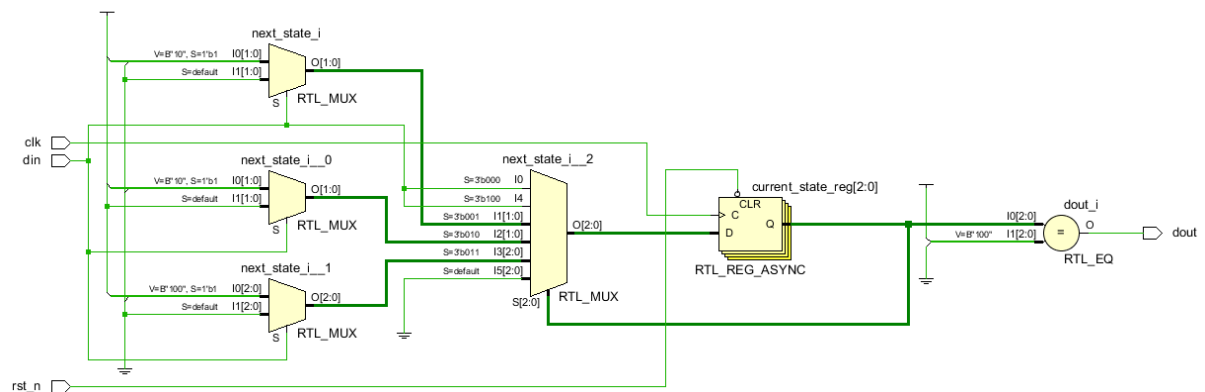
60     send_bit(0);
61
62     #40;
63     $finish;
64 end
65
66 initial begin
67     $dumpfile("dump.vcd");
68     $dumpvars(0, tb_seq_detector_1101_moore);
69 end
70
71 endmodule

```

Simulation Waveforms:



Schematic Diagram:



Task 2:

Objective:

Implement **Traffic Lights** using FSM in SystemVerilog.

Design Code:

```

1 module traffic_light_fsm (
2     input  logic clk,
3     input  logic rst_n,
4     output logic red,
5     output logic yellow,
6     output logic green
7 );
8
9     typedef enum logic [1:0] {
10         S_RED    = 2'b00,
11         S_GREEN  = 2'b01,
12         S_YELLOW = 2'b10
13     } state_t;
14
15     state_t current_state, next_state;
16
17     always_ff @(posedge clk or negedge rst_n) begin
18         if (!rst_n) begin
19             current_state <= S_RED;
20         end else begin
21             current_state <= next_state;
22         end
23     end
24
25     always_comb begin
26         case (current_state)
27             S_RED:    next_state = S_GREEN;
28             S_GREEN:  next_state = S_YELLOW;
29             S_YELLOW: next_state = S_RED;
30             default:  next_state = S_RED;
31         endcase
32     end
33
34     always_comb begin
35         red    = 1'b0;
36         yellow = 1'b0;
37         green  = 1'b0;
38
39         case (current_state)
40             S_RED:    red    = 1'b1;
41             S_GREEN:  green  = 1'b1;
42             S_YELLOW: yellow = 1'b1;
43         endcase
44     end
45
46 endmodule

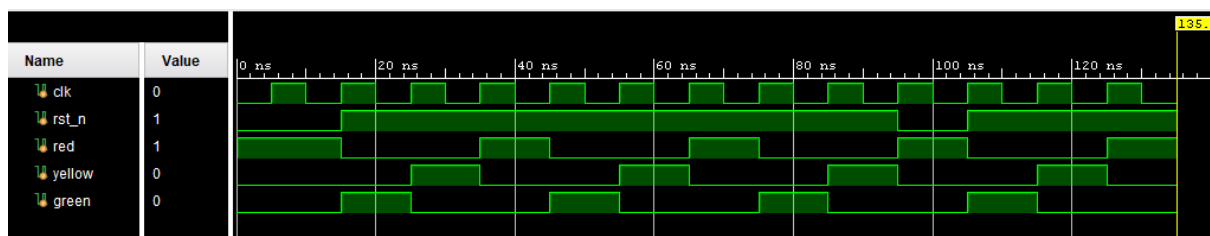
```

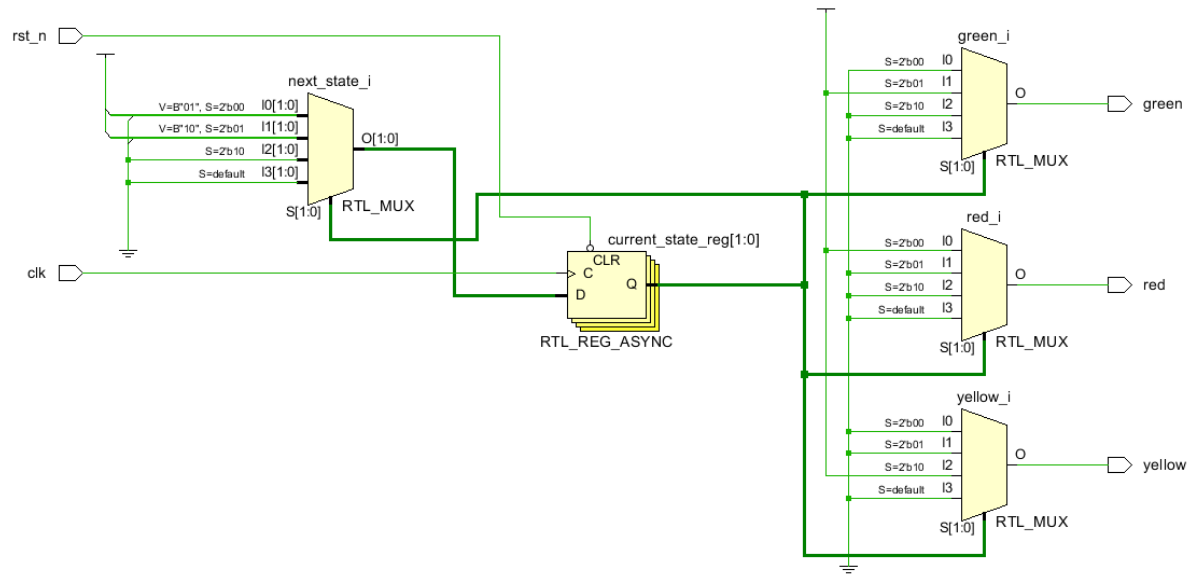
Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_traffic_light_fsm;
4
5      logic clk;
6      logic rst_n;
7      logic red;
8      logic yellow;
9      logic green;
10
11     traffic_light_fsm dut (
12         .clk(clk),
13         .rst_n(rst_n),
14         .red(red),
15         .yellow(yellow),
16         .green(green)
17     );
18
19     always begin
20         #5 clk = ~clk;
21     end
22
23     initial begin
24         clk = 0;
25         rst_n = 0;
26
27         #15 rst_n = 1;
28
29         #80;
30
31         rst_n = 0;
32         #10;
33         rst_n = 1;
34         #30;
35
36         $finish;
37     end
38
39     initial begin
40         $dumpfile("traffic_light_tb.vcd");
41         $dumpvars(0, tb_traffic_light_fsm);
42     end
43
44 endmodule

```

Simulation Waveforms:

Schematic Diagram:

Task 3:

Objective:

Implement a **Vending Machine** using FSM in SystemVerilog.

Design Code:

```

1 module vending_machine (
2     input  logic clk,
3     input  logic rst_n,
4     input  logic nickel,
5     input  logic dime,
6     output logic dispense
7 );
8
9     typedef enum logic [1:0] {
10         S_0c = 2'b00,
11         S_5c = 2'b01,
12         S_10c = 2'b10,
13         S_15c = 2'b11
14     } state_t;
15
16     state_t current_state, next_state;
17
18     always_ff @(posedge clk or negedge rst_n) begin
19         if (!rst_n) begin
20             current_state <= S_0c;
21         end else begin
22             current_state <= next_state;
23         end
24     end
25
26     always_comb begin
27         next_state = current_state;
28
29         case (current_state)
30             S_0c: begin
31                 if (nickel)    next_state = S_5c;
32                 else if (dime) next_state = S_10c;
33             end
34
35             S_5c: begin
36                 if (nickel)    next_state = S_10c;
37                 else if (dime) next_state = S_15c;
38             end
39
40             S_10c: begin
41                 if (nickel || dime) next_state = S_15c;
42             end
43
44             S_15c: begin
45                 next_state = S_0c;
46             end
47
48             default: next_state = S_0c;
49         endcase
50     end
51
52     assign dispense = (current_state == S_15c);
53
54 endmodule

```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_vending_machine;
4
5      logic clk;
6      logic rst_n;
7      logic nickel;
8      logic dime;
9      logic dispense;
10
11     vending_machine dut (
12         .clk(clk),
13         .rst_n(rst_n),
14         .nickel(nickel),
15         .dime(dime),
16         .dispense(dispense)
17     );
18
19     always begin
20         #5 clk = ~clk;
21     end
22
23     task insert_nickel();
24         @(posedge clk);
25         nickel = 1;
26         #1;
27         nickel = 0;
28     endtask
29
30     task insert_dime();
31         @(posedge clk);
32         dime = 1;
33         #1;
34         dime = 0;
35     endtask
36
37     initial begin
38         clk = 0;
39         rst_n = 0;
40         nickel = 0;
41         dime = 0;
42
43         #15 rst_n = 1;
44         #5;
45
46         insert_nickel();
47         insert_nickel();
48         insert_nickel();
49         @(posedge clk);
50
51         #20;
52
53         insert_dime();
54         insert_nickel();
55         @(posedge clk);
56
57         #20;

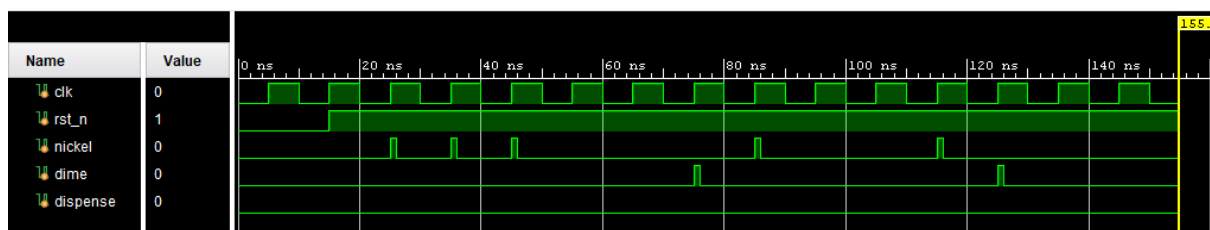
```

```

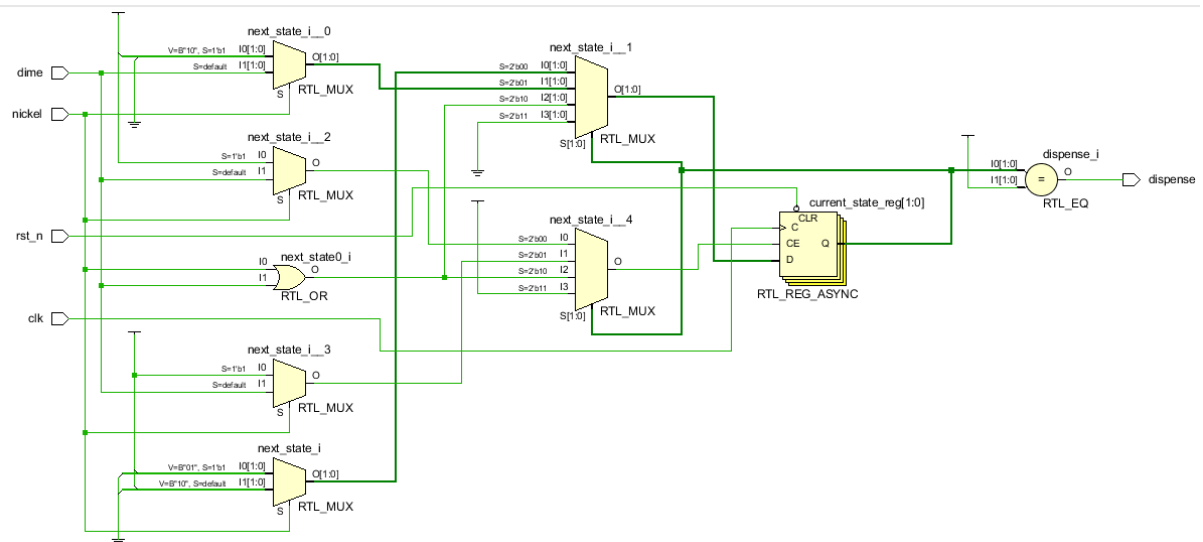
58
59     insert_nickel();
60     insert_dime();
61     @(posedge clk);
62
63     #20;
64     $finish;
65 end
66
67 initial begin
68     $dumpfile("vending_machine_tb.vcd");
69     $dumpvars(0, tb_vending_machine);
70 end
71
72 endmodule

```

Simulation Waveforms:



Schematic Diagram:



Task 4:

Objective:

Implement an 8-bit **FIFO Buffer** in SystemVerilog.

Design Code:

```

1 module fifo_8bit (
2     input logic      clk,
3     input logic      rst_n,
4     input logic      wr_en,
5     input logic      rd_en,
6     input logic [7:0] data_in,
7     output logic [7:0] data_out,
8     output logic      full,
9     output logic      empty
10 );
11
12     parameter DEPTH = 8;
13
14     logic [7:0] memory [DEPTH-1:0];
15
16     logic [2:0] wr_ptr;
17     logic [2:0] rd_ptr;
18     logic [3:0] count;
19
20     assign empty = (count == 0);
21     assign full  = (count == DEPTH);
22
23     always_ff @(posedge clk or negedge rst_n) begin
24         if (!rst_n) begin
25             wr_ptr  <= 3'b0;
26             rd_ptr  <= 3'b0;
27             count   <= 4'b0;
28             data_out <= 8'b0;
29         end else begin
30
31             if (wr_en && !full) begin
32                 memory[wr_ptr] <= data_in;
33                 wr_ptr         <= wr_ptr + 1;
34             end
35
36             if (rd_en && !empty) begin
37                 data_out <= memory[rd_ptr];
38                 rd_ptr   <= rd_ptr + 1;
39             end
40
41             if (wr_en && !full && !(rd_en && !empty)) begin
42                 count <= count + 1;
43             end else if (rd_en && !empty && !(wr_en && !full)) begin
44                 count <= count - 1;
45             end
46         end
47     end
48
49 endmodule

```

Test Bench Code:

```

1  `timescale 1ns/1ps
2
3  module tb_fifo_8bit;
4
5      logic        clk;
6      logic        rst_n;
7      logic        wr_en;
8      logic        rd_en;
9      logic [7:0]  data_in;
10     logic [7:0]  data_out;
11     logic        full;
12     logic        empty;
13
14     fifo_8bit dut (
15         .clk(clk),
16         .rst_n(rst_n),
17         .wr_en(wr_en),
18         .rd_en(rd_en),
19         .data_in(data_in),
20         .data_out(data_out),
21         .full(full),
22         .empty(empty)
23     );
24
25     always #5 clk = ~clk;
26
27     initial begin
28         clk = 0;
29         rst_n = 0;
30         wr_en = 0;
31         rd_en = 0;
32         data_in = 8'h0;
33
34         #15 rst_n = 1;
35         #5;
36
37         write_data(8'hA1);
38         write_data(8'hB2);
39         write_data(8'hC3);
40         #10;
41
42         read_data();
43         read_data();
44         read_data();
45         #10;
46
47         for (int i = 1; i <= 8; i++) begin
48             write_data(i);
49         end
50         #10;
51
52         write_data(8'hFF);
53         #10;
54
55         for (int i = 1; i <= 8; i++) begin
56             read_data();
57         end
58         #10;

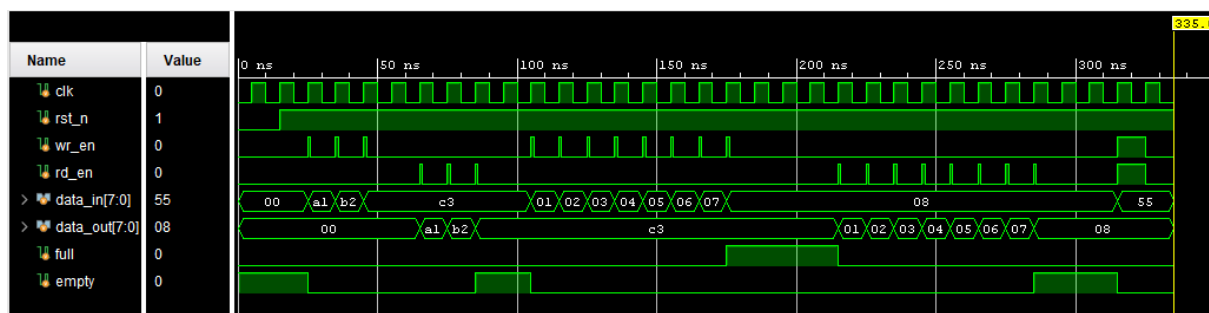
```

```

59
60     read_data();
61     #10;
62
63     @(posedge clk);
64     wr_en = 1;
65     rd_en = 1;
66     data_in = 8'h55;
67     #10;
68     wr_en = 0;
69     rd_en = 0;
70     #10;
71
72     $finish;
73 end
74
75 task write_data(input logic [7:0] val);
76     @(posedge clk);
77     if (!full) begin
78         wr_en = 1;
79         data_in = val;
80     end else begin
81     end
82     #1;
83     wr_en = 0;
84 endtask
85
86 task read_data();
87     @(posedge clk);
88     if (!empty) begin
89         rd_en = 1;
90         #1;
91     end else begin
92     end
93     rd_en = 0;
94 endtask
95
96 initial begin
97     $dumpfile("fifo_8bit_tb.vcd");
98     $dumpvars(0, tb_fifo_8bit);
99 end
100
101 endmodule

```

Simulation Waveforms:



Schematic Diagram:

